

Connexion application Java & base de données

Introduction au projet 'Dressing' & Positionnement des outils
Focus sur la modélisation de l'héritage en BD

Géraldine Del Mondo

Département Informatique et Technologies de l'Information

Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Plan du cours

1 Présentation du projet

- Contexte
- Logistique

2 Outils de mise en œuvre

- Positionnement
- Outils de mise en œuvre
- Point sur l'héritage

3 Conclusion

PRÉSENTATION DU PROJET : CONTEXTE

Sujet : gestion d'un dressing [1]



Sujet : gestion d'un dressing [2]



Concevoir et implanter..

- la base de données
- l'application

.. permettant de..

- gérer un dressing
(ajout/suppression etc)
- en vue de faire de l'aide à la décision
(choix des tenues selon différents critères)

.. en utilisant..

- Postgresql
- Data Mapper/ OrmLite
- java, JDBC

PRÉSENTATION DU PROJET : LOGISTIQUE

Logistique

TD 1 (1h30) :

- ➊ Réalisation du modèle E-A, discussion autour des choix de conception
- ➋ À partir du corrigé → passage au modèle relationnel
- ➌ Discussion sur les Cls à mettre en œuvre, point sur le modèle physique
- ➍ Présentation du modèle (simplifié) qui sera utilisé pour le TP

TP 1 (1h30) :

- ➊ Complétion de la base de données fournie
- ➋ Mise en oeuvre de l'héritage selon les deux méthodes présentées en cours

Logistique

TP 2 (3h) :

- ❶ Complétion des classes java fournies (1/2), premier pas en JDBC
- ❷ Réalisation de fonctionnalités simples (ajout/suppression de tuples, affichage de tables, appel de procédure stockée via l'application
- ❸ CIs triggers gestion des CIs référentielles “manquantes” (devraient théoriquement être mise en place au départ !)

TP 3 (3h00 et +) :

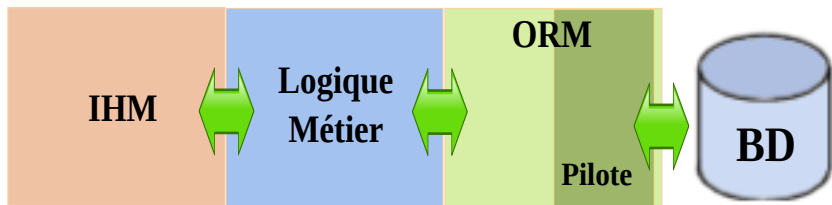
- ❶ Complétion des classes java fournies (2/2), re-structuration de l'architecture java du projet (simplification), mise en place de Data Mapper
- opérations CRUD sur les classes implémentées

TP 4 (3h et -) :

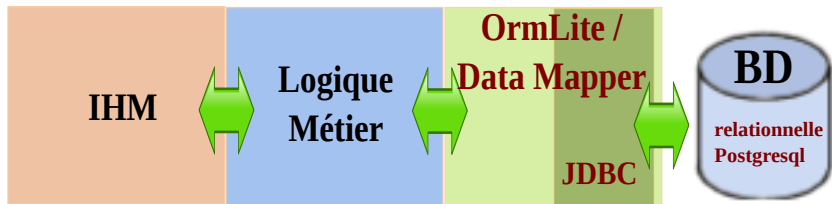
- ❶ L'équivalent du TP3 avec un ORM

OUTILS DE MISE EN ŒUVRE : POSITIONNEMENT

Posgresql, JDBC, patterns Mapper & cie



Posgresql, JDBC, patterns Mapper & cie



OUTILS DE MISE EN ŒUVRE

JDBC

Java DataBase Connectivity

- bibliothèque d'interfaces et de classes
- communication avec un SGBDR
- manipulation de requêtes SQL

Drivers

Il existe différents drivers JDBC pour différentes bases de données

ORM : Object Relational Mapping

Objectif

Cacher la façon dont sont stockées les données à la couche métier

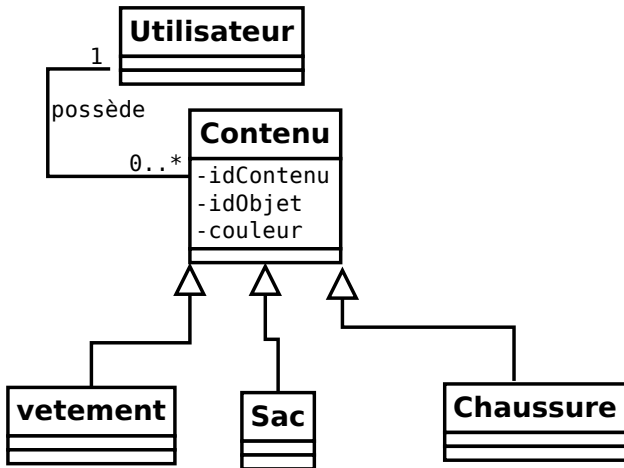
Moyen

- Deux couches : ORM et métier
- La mise en oeuvre de l'ORM peut prendre principalement 2 formes : via un logiciel ORM ou "à la main"

POINT SUR L'HÉRITAGE

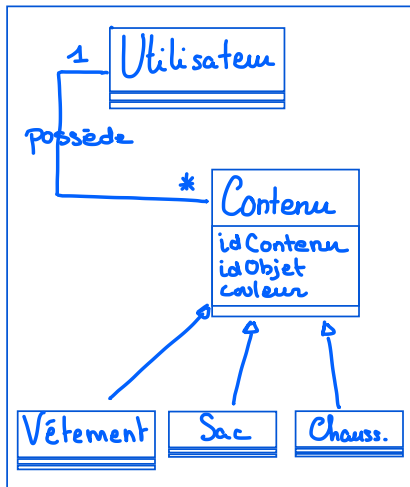
Exemple d'application : extrait du diagramme de classe

→ Que faire persister ?



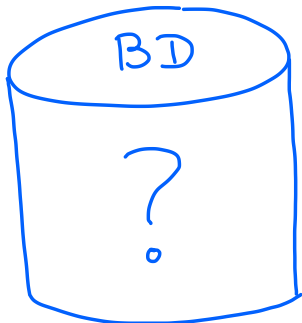
Positionnement logique métier vs. BD

Logique métier



programme (Java)

→ Que faire persister ?

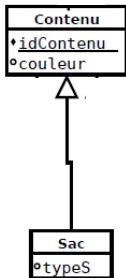


Point technique BD : l'héritage - version via les tables filles

Rappel

Nous avons vu qu'une manière de modéliser l'héritage était de passer par les tables filles.

EXEMPLE

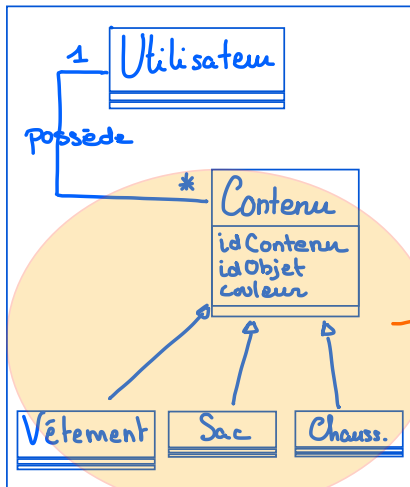


```
CREATE TABLE SAC (  
  idContenu SERIAL PRIMARY KEY,  
  couleur VARCHAR(20),  
  typeS VARCHAR(20))
```

→ Quel que soit le SGBD utilisé, il est toujours possible de faire de cette manière

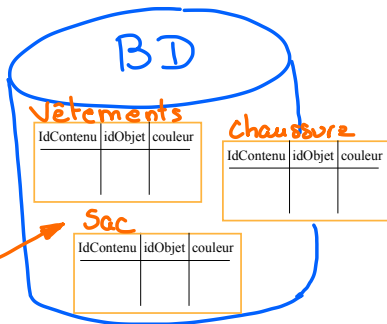
Point technique BD : l'héritage - version via les tables filles

Logique métier

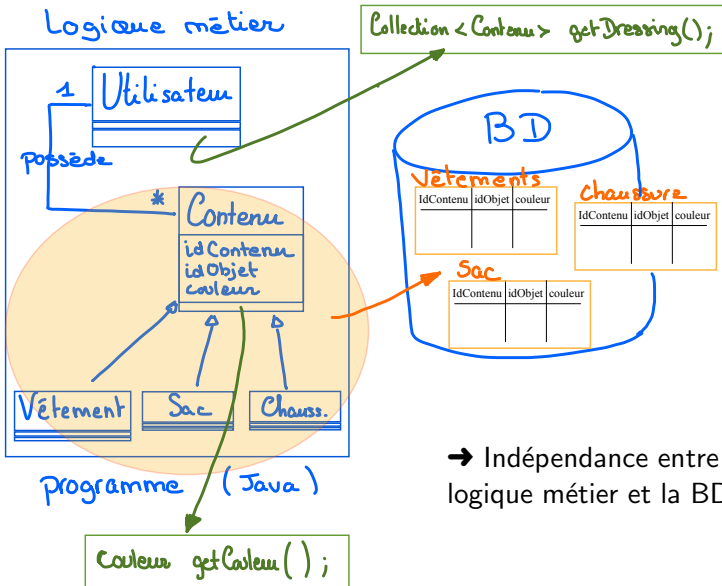


programme (Java)

→ Que faire persister ?

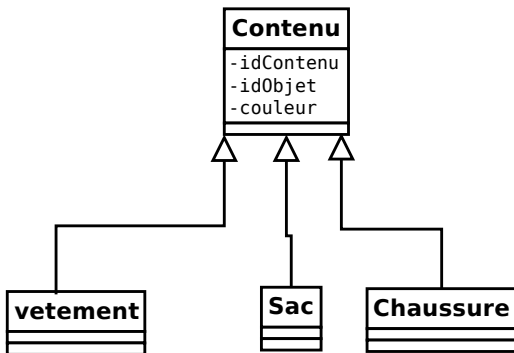


Méthodes de la logique métier



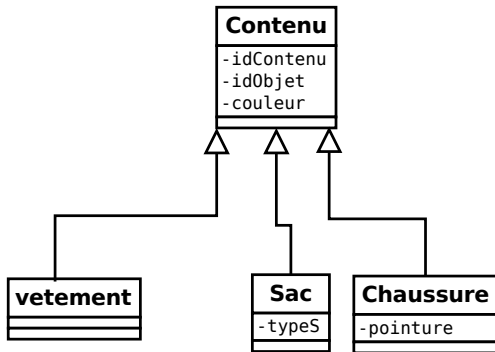
Point technique BD : l'héritage - version via les tables filles

- Quelle requête pour récupérer l'ensemble du contenu du dressing ?



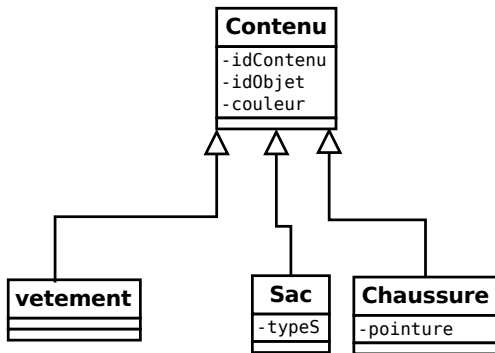
Point technique BD : l'héritage - version via les tables filles

- Quelle requête pour récupérer l'ensemble du contenu du dressing ?
- Quel impact d'ajouter une catégorie de contenu pour la logique métier ?



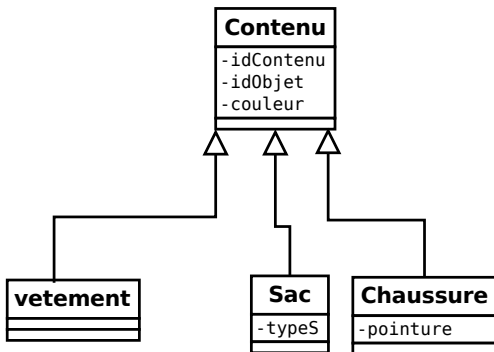
Point technique BD : l'héritage - version via les tables filles

- Quelle requête pour récupérer l'ensemble du contenu du dressing ?
- Quel impact d'ajouter une catégorie de contenu pour la logique métier ?



Point technique BD : l'héritage - version via les tables filles

- Quels sont les deux points notables concernant les Cls relative à l'idContenu dans les tables filles ?



L'héritage via les tables filles

Avantage

Simple à mettre en oeuvre a priori (modulo les CIs défaillantes)

Inconvénients

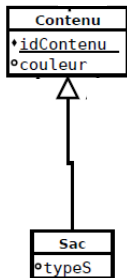
- ❶ On perd certains concepts logiques vis à vis du modèle conceptuel
- ❷ Récupérer l'ensemble de l'information des tables filles n'est pas toujours simple et l'ajout de tables filles modifie la logique métier
- ❸ Certaines contraintes de clés étrangères pour les tables filles peuvent poser des problèmes de mise en oeuvre, il faudra compenser par des triggers.

L'héritage en Postgresql en utilisant INHERITS

Alternative

Une autre façon de modéliser l'héritage est d'utiliser le mot clé INHERITS de Posgresql.

EXEMPLE

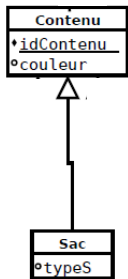


```
CREATE TABLE CONTENU (  
  idContenu SERIAL PRIMARY KEY,  
  couleur VARCHAR(20));
```

```
CREATE TABLE SAC (  
  typeS VARCHAR(20),  
)INHERITS(CONTENU);
```

- Quelle requête pour récupérer l'ensemble du contenu du dressing ?

L'héritage en Postgresql en utilisant INHERITS



```
SELECT * FROM CONTENU;
```

idcontenu	couleur
1	Bleu
2	Rouge
3	Rouge

```
SELECT * FROM SAC;
```

idcontenu	couleur	types
1	Bleu	Sac de sport
2	Rouge	Sac à dos
3	Rouge	Pochette

Remarque : si on ne veut accéder qu'aux données de la table mère on utilisera ONLY
(cas de l'inclusion stricte) `SELECT * FROM ONLY CONTENU;`

L'héritage en Postgresql en utilisant INHERITS

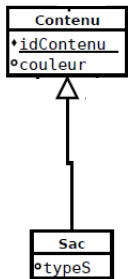
Avantages

- ➊ Plus proche de la modélisation logique (on ne perd pas le concept général)
- ➋ Récupérer l'ensemble de l'information des tables filles est possible en une requête (néanmoins complexe si on ne veut pas que les attributs de la table mère)
- ➔ Potentiellement pas d'impact sur la logique métier lors de l'ajout d'une table fille

Mais... des inconvénients...

L'héritage en Postgresql en utilisant INHERITS

ILLUSTRATION “INCONVÉNIENT” 1



```
INSERT INTO contenu (couleur, typeS) VALUES
      ('Fushia', 'Sac à dos');
```

ERROR: column "typeS" of relation "contenu" does not exist

```
INSERT INTO Contenu (couleur) VALUES ('Fushia');
INSERT 0 1
```

➔ Ce tuple n'a pas été inséré dans Sac, pour le trouver :

```
select * from only contenu ;
```

Cet “inconvenient” ne doit pas se produire (on insère a priori jamais via la table mère), cet exemple illustre le fonctionnement de INHERITS...

L'héritage en Postgresql en utilisant INHERITS

ILLUSTRATION “INCONVÉNIENT” 1 (SUITE)

On insère toujours via les tables filles...

```
INSERT INTO sac (couleur,types) VALUES ('Marron','Pochette');  
INSERT 0 1
```

➔ Ce tuple est visible via la table CONTENU mais il n'appartient pas à CONTENU (via ONLY).

L'héritage en Postgresql en utilisant INHERITS

CONSÉQUENCE INCONVÉNIENT 1

Dans la table CONTENU : idContenu SERIAL PRIMARY KEY

```
INSERT INTO contenu VALUES (1, 'Bleu');
```

```
SELECT * FROM CONTENU;
```

idcontenu	couleur
1	Bleu
2	Rouge
3	Rouge
1	Bleu

L'héritage en Postgresql en utilisant INHERITS

CONSÉQUENCE INCONVÉNIENT 1

Dans la table CONTENU : idContenu SERIAL PRIMARY KEY

```
INSERT INTO contenu VALUES (1, 'Bleu');
```

```
SELECT * FROM CONTENU;
```

idcontenu	couleur
1	Bleu
2	Rouge
3	Rouge
1	Bleu

```
SELECT * FROM ONLY CONTENU;
```

idcontenu	couleur
1	Bleu

L'héritage en Postgresql en utilisant INHERITS

CONSÉQUENCE INCONVÉNIENT 1 (SUITE)

Dans la table CONTENU : idContenu SERIAL PRIMARY KEY

```
SELECT * FROM ONLY CONTENU;
```

idcontenu	couleur
1	Bleu

```
INSERT INTO contenu VALUES (1, 'Rouge');
```

```
ERROR: duplicate key value violates unique constraint  
"contenu_pkey"
```

```
DETAIL: Key (idcontenu)=(1) already exists.
```

L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CIs (1/3)

On insère toujours via les tables filles... et il vaut mieux auto-générer la clé!

```
INSERT INTO sac (idcontenu, couleur,types) VALUES (3,  
'Vert','Pochette');
```

```
INSERT 0 1
```

```
INSERT INTO sac (idcontenu, couleur,types) VALUES (3,  
'Rouge','Sac à dos');
```

```
INSERT 0 1
```

➔ Pas d'héritage de la contrainte de clé primaire dans les tables filles !

L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CIs (2/3)

Dans la table CONTENU : idContenu SERIAL PRIMARY KEY

Si dans une autre table on a : idContenu REFERENCES CONTENU

```
SELECT * FROM ONLY CONTENU;
```

```
idcontenu | couleur
```

```
-----+-----
```

```
1 | Bleu
```

```
9 | Rouge
```

```
SELECT * FROM CONTENU;
```

```
idcontenu | couleur
```

```
-----+-----
```

```
1 | Bleu
```

```
2 | Rouge
```

```
3 | Rouge
```

```
9 | Rouge
```

```
INSERT INTO AUTRE_TABLE (idContenu) values (2);
```

```
ERROR: insert or update on table "autre_table" violates
foreign key constraint "autre_table_idcontenu_fkey"
```

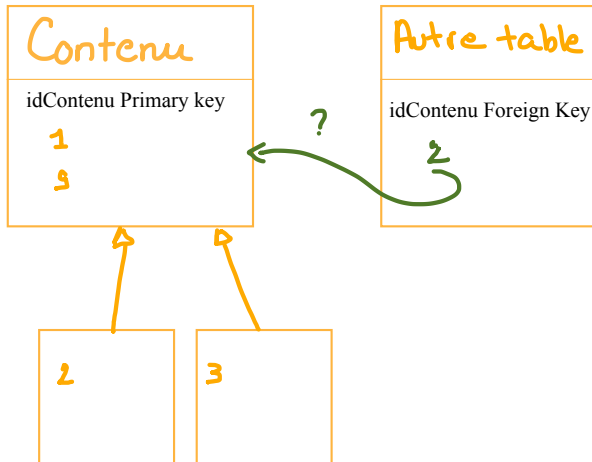
```
DETAIL:Key (idcontenu)=(2) is not present in table "contenu".
```

```
INSERT INTO AUTRE_TABLE (idContenu) values (9);
```

```
INSERT 0 1
```

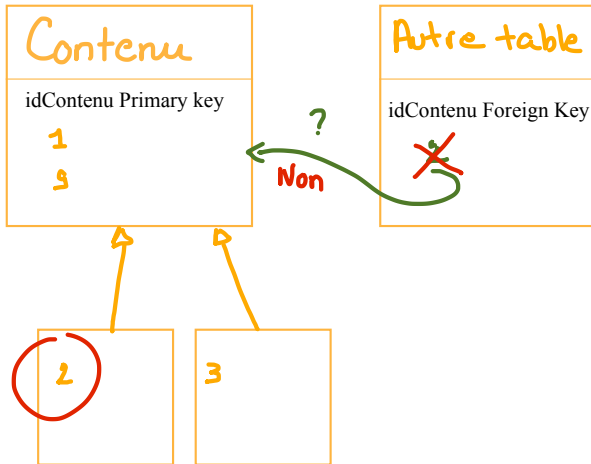
L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CIs (2/3)



L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CONTRAINTES D'INTÉGRITÉ (2/3)



L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CIs (3/3)

Contenu
-idContenu
-couleur
-idDressing

- idContenu SERIAL PRIMARY KEY
- Ajoutons l'attribut : idDressing
REFERENCES DRESSING

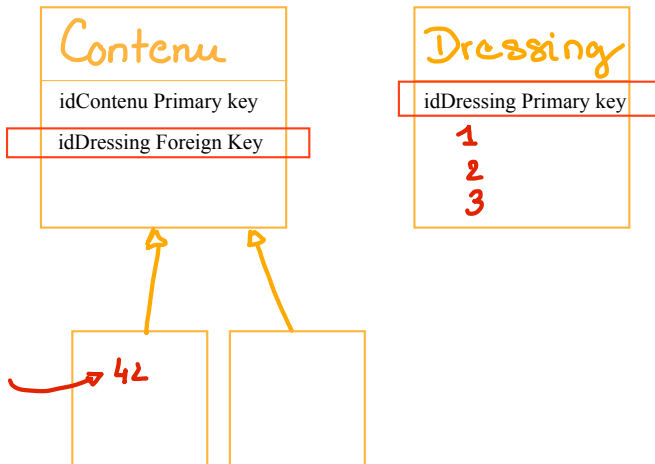
Rien n'empêche de faire :

```
INSERT INTO SAC (idDressing, couleur, typeS) values (  
42, 'Bleu', 'Sac à dos');
```

et ce même si idDressing = 42 n'existe pas dans la table DRESSING.

L'héritage en Postgresql en utilisant INHERITS

UN VRAI PROBLÈME : TRANSMISSION DES CIs (3/3)



L'héritage & Postgresql : synthèse des inconvénients

Inconvénients

- ❶ Le fait d'insérer dans la table mère ne propage pas les données dans la table fille (et réciproquement). Ceci étant, la logique métier n'est pas sensée (en général) permettre une insertion via la table mère.
- ❷ Si toutes les contraintes de vérification et toutes les contraintes NOT NULL sur une table parente sont automatiquement héritées par les tables enfants, les autres types de contraintes (unicité, clé primaire, clé étrangère) ne sont pas hérités !

L'héritage & Postgresql : en général et dans le projet

En général

- L'héritage en Postgresql n'est pas très bien géré* (cf. remarques des slides précédents), mais le principe du partitionnement créé peut être utile dans certains contextes de modélisation. Il faut néanmoins garder à l'esprit les limitations que cela implique et s'efforcer de les gérer au mieux.
- La solution qui consiste à gérer l'héritage dans les tables filles est une alternative qui fonctionne dans tous les cas mais qui a d'autres inconvénients.
- Il en existe d'autres...

Dans le projet

→ via INHERITS, ce qui donnera l'occasion de faire les triggers nécessaires pour vérifier les contraintes de clés étrangères :-)

* "Although inheritance is frequently useful, it has not been integrated with unique constraints or foreign keys, which limits its usefulness", doc Postgresql 10

CONCLUSION

Quelques conseils (généraux)

Pensez votre projet **avant** de vous lancer, autrement dit :

- Réalisez la logique métier de votre application (Faîtes a minima un diagramme de classe)
- Réalisez la base de données (voir slide suivant)
- Implantez progressivement...

→ **Pensez votre logique métier indépendamment de votre persistance !**

Le processus théorique à suivre pour réaliser une base de données ressemble à :

SCHÉMA LOGIQUES

- 1 Récolte des informations réelles permettant de concevoir le schéma de très haut niveau
- 2 Détermination des Dfs, puis CIs (domaine etc.), droits...
- 3 Réalisation du schéma de très haut niveau (y compris schéma externe)
- 4 Passage de ce schéma de très haut niveau à un schéma de haut niveau (schéma externe → vues)
- 5 Normalisation/décomposition des relations (en général en partie dénormalisé par la suite pour des questions de performance)

SCHÉMA PHYSIQUE

- 6 Quel est le volume de données à traiter ? → faut-il définir des index ?
Analyse des "types" de requêtes qui vont être mises en œuvre sur la base
Accès fréquents à quels attributs ? → index primaires/secondaires
- 7 Quelle utilisation de la base ?
Consultation ou modification des données ? Combien d'utilisateurs ?
Modification des données (intensive) → gestion de la concurrence
- 8 APPLIS : choix du mapping

- [1] Aménagement-dressing-moderne-idees <http://deavita.fr>
- [2] The Stormtrooper dressing room for Star Wars : The Force Awakens
Lucasfilm 2015
- [3] Dressing, the costumes of star wars, a galaxy, Trisha Biggar, 2005
- [4] <http://docs.postgresql.fr/9.3/ddl-inherit.html>